HDOC Day-59

Longest Substring Without Repeating Characters in C

Aim

To write a C program using strings, pointers, user-defined functions, arrays, and nested loops to determine the length of the longest contiguous substring with no repeated characters. Uppercase and lowercase letters are distinct; an empty string produces 0.

Algorithm

1. Read the string s using fgets() and remove its trailing newline if present.
2. Pass the string address to longest_unique_substring().
3. Initialize maxLength to 0. For every possible starting index, initialize visited[256] to zeros.
4. Scan forward from the starting index. Stop when a character has already been visited.
5. Otherwise mark the character as visited, increase currentLength, and update maxLength if necessary.
6. Repeat for every starting index, then display maxLength. For an empty string, display 0.

C Program

```
#include <stdio.h>
#include <string.h>

#define MAX 200

void longest_unique_substring(char *s);

int main()
{
    char s[MAX];

    printf("Enter a string: ");
    fgets(s, MAX, stdin);

    /* Remove newline character */
    s[strcspn(s, "\n")] = '\0';

    longest_unique_substring(s);
    return 0;
}

/* Function to find longest unique substring */
void longest_unique_substring(char *s)
{
    int i, j;
    int length;
    int currentLength;
    int maxLength = 0;
    int visited[256];

    length = strlen(s);

    /* Check for empty string */
    if (length == 0)
    {
        printf("Length of Longest Substring: 0\n");
        return;
    }

    /* Select starting position */
    for (i = 0; i < length; i++)
    {
        /* Initialize visited array */
        for (j = 0; j < 256; j++)
        {
            visited[j] = 0;
        }

        currentLength = 0;

        /* Check characters in current substring */
        for (j = i; j < length; j++)
        {
            unsigned char ch = *(s + j);

            /* Stop if character repeats */
            if (visited[ch] == 1)
            {
                break;
            }

            visited[ch] = 1;
            currentLength++;

            /* Update maximum length */
            if (currentLength > maxLength)
            {
                maxLength = currentLength;
```

```
            }
        }
    }

    printf("Length of Longest Substring: %d\n",
           maxLength);
}
```

Explanation

The function **longest_unique_substring(char *s)** receives the string through a pointer. The expression ***(s + j)** accesses the character at index j. For each starting position, the fixed-size visited array tracks which byte values have appeared, so uppercase and lowercase ASCII letters are treated differently.

Example: abcabcbb

Starting index	Longest unique substring	Length
0	abc	3
1	bca	3
2	cab	3
3	abc	3
4	bc	2
5	cb	2
6	b	1
7	b	1

The maximum length is 3. For the input aAbBcC, all six characters are distinct and the result is 6. The nested-loop approach takes O(n^2) time and O(1) auxiliary space (fixed 256-element array).

Test Case Table

Case	Input string	Expected length
1	abcabcbb	3
2	bbbbb	1
3	pwwkew	3
4	abcdef	6
5	aAbBcC	6
6	abba	2
7	dvdf	3
8	a	1
9	(empty string)	0
10	AaAa	2

Sample Outputs

Repeating characters

```
Enter a string: abcabcbb
Length of Longest Substring: 3
```

All characters identical

```
Enter a string: bbbbb
Length of Longest Substring: 1
```

Mixed repeating characters

```
Enter a string: pwwkew
Length of Longest Substring: 3
```

Uppercase and lowercase

```
Enter a string: aAbBcC
Length of Longest Substring: 6
```

Empty string

```
Enter a string:
Length of Longest Substring: 0
```

Result

The program uses a user-defined function, pointers, arrays, nested loops, and conditional statements to calculate the length of the longest contiguous substring with no repeated characters. It handles case-sensitive comparisons, repeated characters, single-character inputs, and empty strings.